



FORMA

PARCIAL 1 · APLICACIONES WEB 1

Arquitectura semántica HTML5 y modelo de caja CSS

Investigación técnica y demostración práctica

ALUMNA	Guadalupe Astudillo Pini
INSTITUCIÓN	Colegio Universitario IES
MATERIA	Aplicaciones Web 1
TEMA ELEGIDO	Arquitectura semántica HTML5 y modelo de caja CSS
FECHA	Octubre de 2026

ÍNDICE DE CONTENIDOS

Contenido

1. Introducción	3
2. Arquitectura semántica HTML5	3
3. Modelo de caja CSS	3
4. Relación entre HTML y CSS	3
5. Demostración práctica	4
5.1 Arquitectura semántica	4
5.2 Modelo de caja	4
5.3 Prueba de herencia de box-sizing	5
5.4 Progresión del trabajo	5
6. Conclusiones	6
7. Fuentes de consulta	6

OBJETIVO DEL TRABAJO

Vincular los conceptos teóricos del Tema 1 con una demostración funcional en HTML y CSS, utilizando vocabulario técnico preciso, estructura semántica y una comparación visual del modelo de caja.

MARCO TEÓRICO

1. Introducción

HTML y CSS cumplen funciones distintas pero complementarias: HTML define la estructura y el significado del contenido, y CSS controla su presentación y el espacio que ocupa cada elemento. Este trabajo relaciona la arquitectura semántica de HTML5 con el modelo de caja de CSS mediante una demostración funcional.

2. Arquitectura semántica HTML5

El marcado semántico utiliza elementos cuyo nombre describe la función del contenido: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>` y `<footer>`. A diferencia de los contenedores genéricos como `<div>`, construyen una jerarquía más clara. Esto favorece la accesibilidad, porque los lectores de pantalla pueden identificar regiones del documento y facilitar la navegación, y además aporta contexto a los motores de búsqueda. La semántica, por sí sola, no define la apariencia visual.

Concepto clave: una etiqueta semántica describe qué función cumple un bloque dentro del documento, no cómo debe verse.

3. Modelo de caja CSS

Todo elemento renderizado puede analizarse con el modelo de caja: contenido, padding, border y margin. Con el valor inicial `content-box`, `width` y `height` definen solo el área de contenido, por lo que padding y border se suman al tamaño declarado. Con `box-sizing: border-box`, ambos quedan incluidos en el ancho y alto declarados; el margin siempre queda por fuera. En la demostración, dos cajas con `width: 300px`, `padding: 30px` y `border: 5px` miden 370px con `content-box` ($300 + 60 + 10$) y 300px con `border-box`. La propiedad `box-sizing` no se hereda: debe declararse en cada elemento o mediante una regla general.

4. Relación entre HTML y CSS

HTML aporta la estructura y CSS la toma como base mediante selectores. En la demostración, dos elementos `<article>` con el mismo contenido y las mismas medidas se comportan distinto porque cada uno recibe una clase diferente (`.caja-content` y `.caja-border`). La distribución de los bloques se resuelve con Flexbox (`display: flex`), según la guía de CSS-Tricks. HTML da significado y orden; CSS define dimensiones, distribución y apariencia.

Vocabulario técnico aplicado

- marcado semántico
- jerarquía de contenidos
- accesibilidad
- modelo de caja
- content-box
- border-box
- padding
- border
- ancho calculado (computed width)
- herencia CSS
- selector de clase
- Flexbox

DEMOSTRACIÓN CODIFICADA

5. Demostración práctica

La práctica se resolvió con un archivo `index.html` y una hoja `estilos.css` separados, indentados y comentados. La página muestra el concepto investigado mediante tres demostraciones visuales: la jerarquía semántica HTML5, la diferencia entre `content-box` y `border-box`, y la prueba de que `box-sizing` no se hereda.

5.1 Arquitectura semántica

El esquema reproduce la estructura real de la página: el `<nav>` está dentro del `<header>`, y el `<article>` dentro de la `<section>`, que a su vez está dentro de `<main>`. Los `<div>` se usan únicamente para dibujar el esquema; la estructura de la página se arma con elementos semánticos.

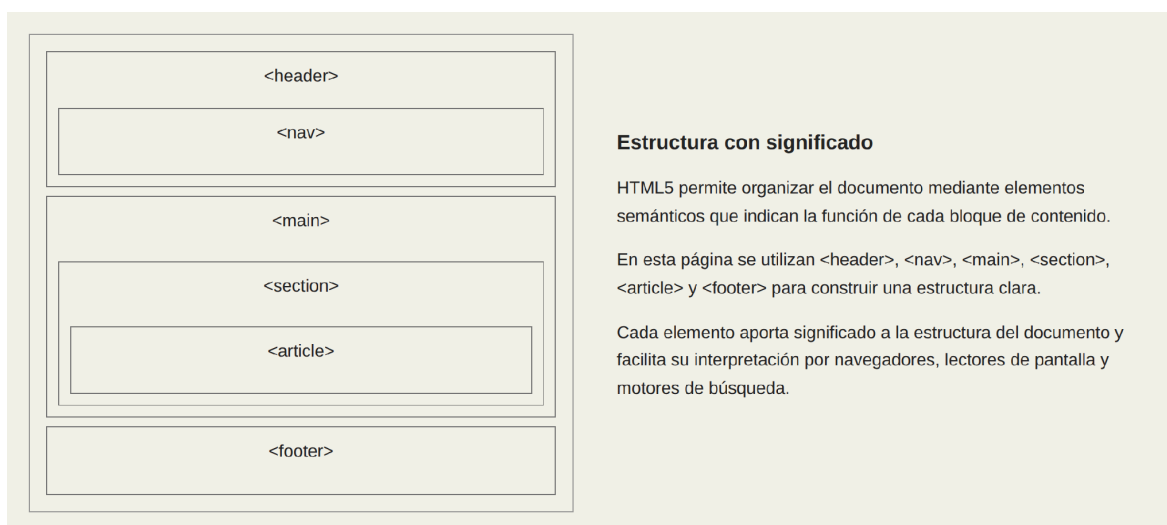


Figura 1. Representación visual de `<header>`, `<nav>`, `<main>`, `<section>`, `<article>` y `<footer>`.

5.2 Modelo de caja

Las dos cajas comparten `width`, `padding` y `border`. En cuanto al cálculo de dimensiones, la propiedad que cambia es `box-sizing`. Los colores se utilizan únicamente para diferenciarlas visualmente: terracota para la caja por defecto (370px) y verde para la alternativa (300px).



Figura 2. Comparación de comportamiento por defecto (`content-box`) y modelo alternativo (`border-box`).

5.3 Prueba de herencia de box-sizing

Para comprobar que `box-sizing` no se hereda, el elemento padre declara `border-box` con `width: 300px`, `padding: 20px` y `border: 5px`, por lo que su contenido mide 250px ($300 - 40 - 10$). El hijo no declara `box-sizing` y usa `width: 100%`: al volver al valor inicial `content-box`, suma su propio `padding` (30px) y `border` (4px) y llega a 284px, es decir, 34px más que el espacio disponible. Si la propiedad se heredara, el hijo cabría dentro del padre.

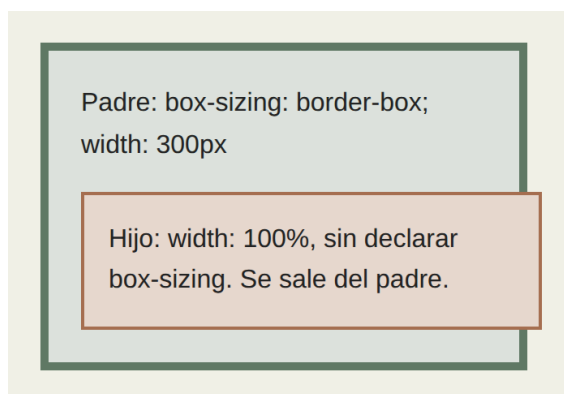


Figura 3. El hijo ocupa 284px sobre un área de contenido disponible de 250px, evidenciando que `box-sizing` no se hereda.

5.4 Progresión del trabajo

El proyecto se desarrolló por etapas, registradas como commits en el repositorio Parcial1AstudilloAW:

1. Estructura semántica y modelo de caja.
2. Demostración visual de la arquitectura semántica HTML5.
3. Favicon e identidad visual (marca FORMA).
4. Revisión final: comentarios en el código, corrección ortográfica y prueba de herencia de `box-sizing`.

Repositorio: <https://github.com/astudilloguadalupe/Parcial1AstudilloAW>

CIERRE

6. Conclusiones

La investigación permite comprobar que una página web no depende únicamente de su aspecto visual. Una estructura HTML semántica mejora la organización y aporta significado al contenido, mientras que el modelo de caja de CSS determina cómo se calculan las dimensiones de los elementos. La comparación práctica evidencia que dos elementos con las mismas propiedades declaradas pueden ocupar tamaños diferentes según el valor de `box-sizing`, y que esa propiedad no pasa de padres a hijos. Como criterio de trabajo, se recomienda utilizar etiquetas semánticas cuando exista un elemento apropiado, mantener HTML y CSS separados y aplicar `border-box` de manera consciente cuando se necesite un cálculo de dimensiones más predecible.

7. Fuentes de consulta

MDN Web Docs. *Referencia de elementos HTML.*

<https://developer.mozilla.org/es/docs/Web/HTML/Reference/Elements>

CSS-Tricks. *A Complete Guide to Flexbox.*

<https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

MDN Web Docs. *box-sizing - CSS.*

<https://developer.mozilla.org/es/docs/Web/CSS/box-sizing>

Consultadas el 1 de octubre de 2026 para el desarrollo del Parcial 1.